LAB 6

Code and Branch Coverage Analysis

Vending Machine Test Suite

Student	Shibi R Shetty
SRN	PES2UG24CS467
Repository	`https://github.com/shibishetty77/lab6`
Application under test	vending_machine.py

Submission contents

• Code coverage analysis and terminal evidence

• Branch coverage analysis and terminal evidence

• Test-case source listings for both required test files

• Commands used to run the coverage reports

• Final submission checklist mapped to the student handout

Reported results: code coverage — 100% (15 tests passed); branch coverage — 100% (15 tests passed). The screenshots in this report are the terminal evidence supplied for these runs.

Note: this PDF consolidates the report evidence and test code. The handout also requires the two test files and at least two distinct commits to be present in the GitHub repository.

1. Objective and Results

The purpose of this lab is to use pytest and coverage.py to measure executed statements and decision branches in the vending machine application, then expand the tests to exercise normal, boundary, and error paths.

Metric	Code coverage run	Branch coverage run
Test file	CodeCoverageTest_vending_machine.py	BranchCoverageTest_vending_machine.py
Tests passed	15	15
Statements	30	30
Missed statements	0	0
Branches	Not reported in line-only run	18
Partial branches	Not reported in line-only run	0
Reported coverage	100%	100%

2. Commands Used

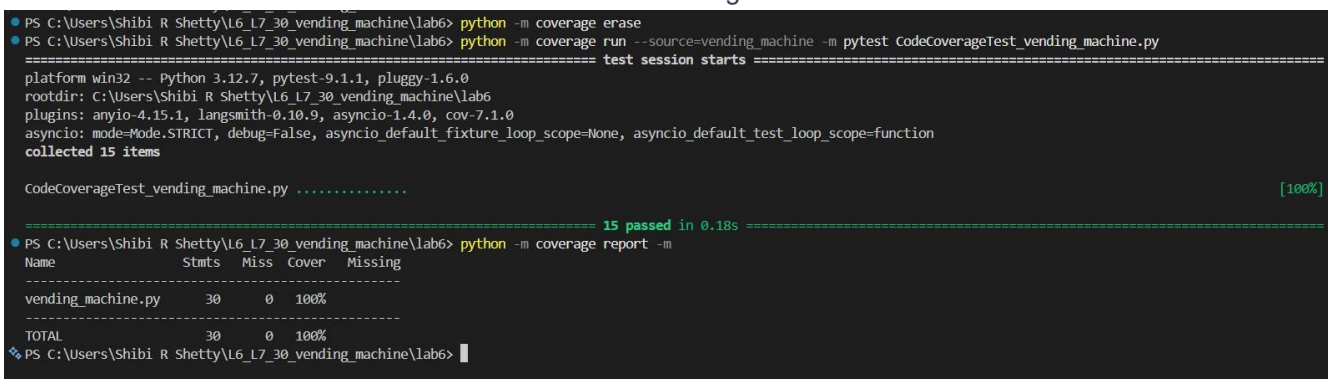
```
# Code coverage python -m coverage erase python -m coverage run --source=vending_machine -m
pytest CodeCoverageTest_vending_machine.py python -m coverage report -m

# Branch coverage python -m coverage erase python -m coverage run --branch --source=vending_machine -m
pytest BranchCoverageTest_vending_machine.py python -m coverage report -m
```

The `--branch` option includes branch measurement. The report is generated after the corresponding test run; `coverage erase` is run before each independent report so results are not mixed.

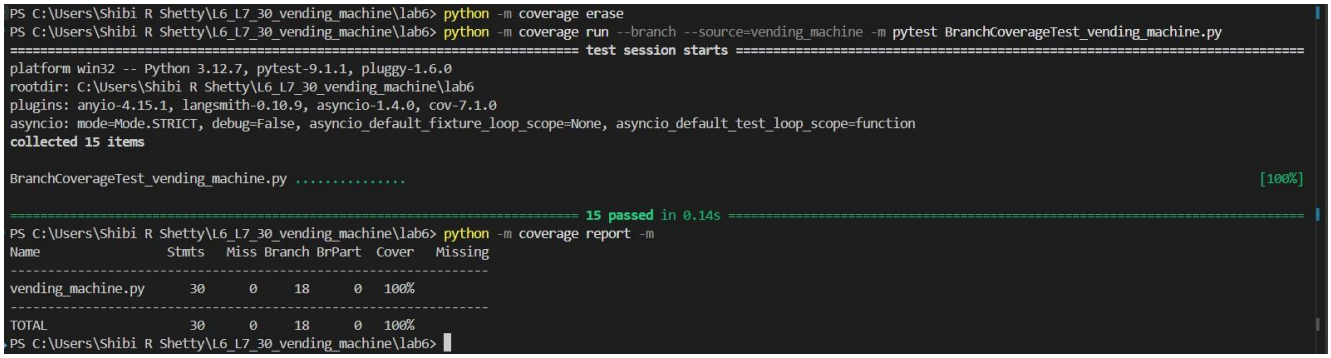
3. Code Coverage — Terminal Evidence

The screenshot shows the code coverage test run: 15 tests passed, 30 statements, 0 missed statements, 100% coverage.



4. Branch Coverage — Terminal Evidence

The screenshot shows the branch coverage test run: 15 tests passed, 30 statements, 0 missed statements, 18 branches, 0 partial branches, and 100% coverage.



5. Code Coverage Test Source

File: CodeCoverageTest_vending_machine.py

```
import pytest

from vending_machine import select_item, make_change, purchase


def test_invalid_item():
    assert select_item("water", {}, 100) == "invalid"


def test_sold_out_item():
    assert select_item("cola", {"cola": 0}, 200) == "sold_out"


def test_missing_stock_item():
    assert select_item("chips", {}, 200) == "sold_out"


def test_insufficient_credit():
    assert select_item("cola", {"cola": 2}, 100) == "insufficient"


def test_valid_selection():
    assert select_item("cola", {"cola": 2}, 150) == "ok"


def test_exact_change_zero():
assert make_change(0) == []


def test_change_using_one_coin():
assert make_change(100) == [100]


def test_change_using_multiple_coins():
    assert make_change(185) == [100, 50, 25, 10]


def test_negative_change_raises_error():     with
pytest.raises(ValueError, match="negative amount"):
make_change(-1)


def test_unrepresentable_change():
assert make_change(3) is None


def test_purchase_invalid_item():
    assert purchase("water", {}, 100) == ("invalid", [])


def test_purchase_sold_out():     assert purchase("cola", {"cola":
0}, 200) == ("sold_out", [])


def test_purchase_insufficient_credit():     assert purchase("cola",
{"cola": 2}, 100) == ("insufficient", [])


def test_successful_purchase_and_stock_update():
stock = {"cola": 2}     status, change =
purchase("cola", stock, 200)

    assert status == "dispensed"
assert change == [50]     assert
stock["cola"] == 1


def test_purchase_without_exact_change():
    stock = {"chips": 1}     assert purchase("chips", stock, 103) ==
("no_exact_change", [])     assert stock["chips"] == 1
```

6. Branch Coverage Test Source

File: BranchCoverageTest_vending_machine.py

```
from vending_machine import select_item, make_change, purchase


def test_credit_below_price():
    assert select_item("chips", {"chips": 1}, 99) == "insufficient"


def test_credit_equal_to_price():
    assert select_item("chips", {"chips": 1}, 100) == "ok"


def test_credit_above_price():
    assert select_item("chips", {"chips": 1}, 150) == "ok"


def test_change_loop_skipped():
assert make_change(0) == []


def test_change_loop_executed():
assert make_change(50) == [50]


def test_remaining_unrepresentable_amount():
assert make_change(1) is None


def test_purchase_rejected_does_not_change_stock():
    stock = {"chips": 2}     result =
purchase("chips", stock, 50)     assert
result == ("insufficient", [])     assert
stock["chips"] == 2


def test_purchase_success_decrements_stock():
stock = {"chips": 2}     result =
purchase("chips", stock, 100)     assert
result == ("dispensed", [])     assert
stock["chips"] == 1


def test_invalid_item_purchase():
    assert purchase("water", {}, 100) == ("invalid", [])


def test_sold_out_item_purchase():     assert purchase("cola",
{"cola": 0}, 200) == ("sold_out", [])


def test_change_exact_multiple_coins():
assert make_change(75) == [50, 25]


def test_change_with_unrepresentable_remainder():
assert make_change(7) is None


def test_purchase_exact_payment():
    stock = {"cola": 1}     result =
purchase("cola", stock, 150)     assert
result == ("dispensed", [])     assert
stock["cola"] == 0


def test_purchase_no_exact_change():
    stock = {"chips": 1}     result =
purchase("chips", stock, 103)     assert
result == ("no_exact_change", [])     assert
stock["chips"] == 1


def test_negative_change():
    import pytest     with pytest.raises(ValueError,
match="negative amount"):
        make_change(-1)
```